

```

        a[j]=v;
    }
} while (inc > 1);
}

```

CITED REFERENCES AND FURTHER READING:

Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §5.2.1.[1]

Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), Chapter 8.

8.2 Quicksort

Quicksort is, on most machines, on average, for large N , the fastest known sorting algorithm. It is a “partition-exchange” sorting method: A “partitioning element” a is selected from the array. Then, by pairwise exchanges of elements, the original array is partitioned into two subarrays. At the end of a round of partitioning, the element a is in its final place in the array. All elements in the left subarray are $\leq a$, while all elements in the right subarray are $\geq a$. The process is then repeated on the left and right subarrays independently, and so on.

The partitioning process is carried out by selecting some element, say the leftmost, as the partitioning element a . Scan a pointer up the array until you find an element $> a$, and then scan another pointer down from the end of the array until you find an element $< a$. These two elements are clearly out of place for the final partitioned array, so exchange them. Continue this process until the pointers cross. This is the right place to insert a , and that round of partitioning is done. The question of the best strategy when an element is equal to the partitioning element is subtle; see Sedgewick [1] for a discussion. (Answer: You should stop and do an exchange.)

For speed of execution, we don’t implement Quicksort using recursion. Thus the algorithm requires an auxiliary array of storage, of length $2 \log_2 N$, which it uses as a push-down stack for keeping track of the pending subarrays. When a subarray has gotten down to some size M , it becomes faster to sort it by straight insertion (§8.1), so we will do this. The optimal setting of M is machine-dependent, but $M = 7$ is not too far wrong. Some people advocate leaving the short subarrays unsorted until the end, and then doing one giant insertion sort at the end. Since each element moves at most seven places, this is just as efficient as doing the sorts immediately, and saves on the overhead. However, on modern machines with a cache hierarchy, there is increased overhead when dealing with a large array all at once. We have not found any advantage in saving the insertion sorts till the end.

As already mentioned, Quicksort’s *average* running time is fast, but its *worst case* running time can be very slow: For the worst case it is, in fact, an N^2 method! And for the most straightforward implementation of Quicksort it turns out that the worst case is achieved for an input array that is already in order! This ordering of the input array might easily occur in practice. One way to avoid this is to use a little random number generator to choose a random element as the partitioning element. Another is to use instead the median of the first, middle, and last elements of the current subarray.

The great speed of Quicksort comes from the simplicity and efficiency of its inner loop. Simply adding one unnecessary test (for example, a test that your pointer has not moved off the end of the array) can almost double the running time! One avoids such unnecessary tests by placing “sentinels” at either end of the subarray being partitioned. The leftmost sentinel is $\leq a$, the rightmost $\geq a$. With the “median-of-three” selection of a partitioning element, we can use the two elements that were not the median to be the sentinels for that subarray.

Our implementation closely follows [1]:

```
sort.h  template<class T>
        void sort(NRvector<T> &arr, Int m=-1)
Sort an array arr[0..n-1] into ascending numerical order using the Quicksort algorithm. arr
is replaced on output by its sorted rearrangement. Normally, the optional argument m should be
omitted, but if it is set to a positive value, then only the first m elements of arr are sorted.
{
    static const Int M=7, NSTACK=64;
    Here M is the size of subarrays sorted by straight insertion and NSTACK is the required
    auxiliary storage.
    Int i,ir,j,k,jstack=-1,l=0,n=arr.size();
    T a;
    VecInt istack(NSTACK);
    if (m>0) n = MIN(m,n);           Use optional argument.
    ir=n-1;
    for (;;) {                       Insertion sort when subarray small enough.
        if (ir-l < M) {
            for (j=l+1;j<=ir;j++) {
                a=arr[j];
                for (i=j-1;i>=l;i--) {
                    if (arr[i] <= a) break;
                    arr[i+1]=arr[i];
                }
                arr[i+1]=a;
            }
            if (jstack < 0) break;
            ir=istack[jstack--];
            l=istack[jstack--];
        } else {
            k=(l+ir) >> 1;           Choose median of left, center, and right el-
            SWAP(arr[k],arr[l+1]);   ements as partitioning element a. Also
            if (arr[l] > arr[ir]) {   rearrange so that a[l] ≤ a[l+1] ≤ a[ir].
                SWAP(arr[l],arr[ir]);
            }
            if (arr[l+1] > arr[ir]) {
                SWAP(arr[l+1],arr[ir]);
            }
            if (arr[l] > arr[l+1]) {
                SWAP(arr[l],arr[l+1]);
            }
            i=l+1;                   Initialize pointers for partitioning.
            j=ir;
            a=arr[l+1];              Partitioning element.
            for (;;) {               Beginning of innermost loop.
                do i++; while (arr[i] < a);   Scan up to find element > a.
                do j--; while (arr[j] > a);   Scan down to find element < a.
                if (j < i) break;             Pointers crossed. Partitioning complete.
                SWAP(arr[i],arr[j]);         Exchange elements.
            }                               End of innermost loop.
            arr[l+1]=arr[j];               Insert partitioning element.
            arr[j]=a;
            jstack += 2;
            Push pointers to larger subarray on stack; process smaller subarray immediately.
        }
    }
}
```

```

        if (jstack >= NSTACK) throw("NSTACK too small in sort.");
        if (ir-i+1 >= j-1) {
            istack[jstack]=ir;
            istack[jstack-1]=i;
            ir=j-1;
        } else {
            istack[jstack]=j-1;
            istack[jstack-1]=l;
            l=i;
        }
    }
}
}

```

As usual, you can move any other arrays around at the same time as you sort `arr`. At the risk of being repetitious:

```

template<class T, class U>
void sort2(NRvector<T> &arr, NRvector<U> &brr)
Sort an array arr[0..n-1] into ascending order using Quicksort, while making the corresponding
rearrangement of the array brr[0..n-1].
{
    const Int M=7,NSTACK=64;
    Int i,ir,j,k,jstack=-1,l=0,n=arr.size();
    T a;
    U b;
    VecInt istack(NSTACK);
    ir=n-1;
    for (;;) {
        if (ir-l < M) {
            for (j=l+1;j<=ir;j++) {
                a=arr[j];
                b=brr[j];
                for (i=j-1;i>=l;i--) {
                    if (arr[i] <= a) break;
                    arr[i+1]=arr[i];
                    brr[i+1]=brr[i];
                }
                arr[i+1]=a;
                brr[i+1]=b;
            }
            if (jstack < 0) break;
            ir=istack[jstack--];
            l=istack[jstack--];
        } else {
            k=(l+ir) >> 1;
            SWAP(arr[k],arr[l+1]);
            SWAP(brr[k],brr[l+1]);
            if (arr[l] > arr[ir]) {
                SWAP(arr[l],arr[ir]);
                SWAP(brr[l],brr[ir]);
            }
            if (arr[l+1] > arr[ir]) {
                SWAP(arr[l+1],arr[ir]);
                SWAP(brr[l+1],brr[ir]);
            }
            if (arr[l] > arr[l+1]) {
                SWAP(arr[l],arr[l+1]);
                SWAP(brr[l],brr[l+1]);
            }
            i=l+1;
            j=ir;
            a=arr[l+1];

```

sort.h

Insertion sort when subarray small enough.

Pop stack and begin a new round of partitioning.

Choose median of left, center, and right elements as partitioning element a. Also rearrange so that $a[l] \leq a[l+1] \leq a[ir]$.

Initialize pointers for partitioning.

Partitioning element.